

Informe — Práctica de Sincronización

Autor: Juan Llorens — JuanLlorensULL **Repositorio:** <https://github.com/JuanLlorensULL/SOA-sincronizacion> **Asignatura:** Sistemas Operativos Avanzados (ULL — ESIT)

1. Alcance

El enunciado plantea una comparativa entre los problemas **productor-consumidor** y **lector-escritor** sobre un mismo ring buffer. Por indicación expresa del profesor, este informe cubre **únicamente productor-consumidor**. Las preguntas de reflexión que aluden al lector-escritor se responden razonando sobre el diseño *que tendría que tener* esa segunda implementación, sin presentar datos experimentales que no se han recogido.

Toda la implementación y los logs viven en `productor-consumidor/`:

- Código fuente: `productor-consumidor/src/` + `productor-consumidor/include/`.
- Diagramas de flujo (Mermaid): `productor-consumidor/docs/diagrama-flujo.md`.
- Log de Valgrind: `productor-consumidor/docs/valgrind.log` — 0 errores, 0 leaks (260 `allocs` / 260 `frees`).
- Logs de los dos casos de tiempos: `productor-consumidor/docs/caso1.log` y `productor-consumidor/docs/caso2.log`.

2. Resumen de la implementación

- **Lenguaje:** C++17 con `std::thread`, `std::mutex` y `std::condition_variable`. Build con GNU Make (`productor-consumidor/Makefile`).
- **Monitor (Buffer):** ring buffer de capacidad fija. La sincronización se resuelve con un único mutex y dos variables de condición:
 - `not_full_` — el productor espera con predicado `count_ < capacity_`.
 - `not_empty_` — los consumidores esperan con predicado `count_ > 0` || `producer_done_`.
- **API del monitor:** `add(int)` para el productor; `remove()` devuelve `std::optional<int>` para los consumidores — `nullopt` indica fin de stream limpio (productor terminado y buffer vacío), evitando centinelas mágicos en el flujo de datos.
- **Terminación:** el productor llama `mark_done()` al acabar, que activa `producer_done_` y hace `notify_all` sobre `not_empty_` para despertar a los 3 consumidores bloqueados.
- **Consumidores:** un mismo functor `Consumer` parametrizado por `id ∈ {1, 2, 3}` hace `dispatch` a la operación correspondiente. Cada uno mantiene su propia copia mutable del vector inicial de 100 enteros y una lista del histórico de números consumidos.
- **Operaciones (puras, en `Operations.hpp/cpp`):**
 - `id=1` — `restar_y_calcular_moda`: resta el número leído a cada elemento in-place y devuelve la moda y los 3 valores más repetidos.
 - `id=2` — `promediar_y_calcular_desviacion`: actualiza `v[i] = (v[i] + leído) / 2` in-place y devuelve la desviación estándar poblacional.
 - `id=3` — `sumatoria_por`: devuelve $(\sum v) \cdot \text{leído}$. No muta el vector.
- **Salida serializada:** un `std::mutex` global declarado inline en `Log.hpp` (`cout_mtx`) protege `std::cout` para que las trazas de los 4 hilos no se entremezclen carácter a carácter.

3. Experimentación: casos de tiempos de espera

Ambos casos se han ejecutado con los parámetros del enunciado: 30 items, buffer de 5 huecos, 1 productor, 3 consumidores. Se mide el wall-clock con `/usr/bin/time`:

Caso	--prod-sleep	--cons-sleep	Cuello de botella	Wall-clock	Items consumidos C1 / C2 / C3
1	100 ms	300 ms	consumidores	3.20 s	10 / 10 / 10
2	300 ms	100 ms	productor	9.00 s	10 / 10 / 10

Modelo teórico (despreciando coste de cómputo y arranque):

- **Caso 1.** Throughput agregado del lado consumidor con 3 hilos a 300 ms cada uno: $3 / 300 \text{ ms} = 1 \text{ item} / 100 \text{ ms}$. El productor produce a la misma tasa (1 item / 100 ms). El sistema queda balanceado y el buffer se sostiene con ocupación baja. Tiempo total esperado: $N \times 100 \text{ ms} = 30 \times 100 \text{ ms} = 3.0 \text{ s}$. Medido: 3.20 s.
- **Caso 2.** El productor produce 1 item cada 300 ms y los consumidores los drenan inmediatamente. El buffer pasa la mayor parte del tiempo casi vacío y los consumidores se bloquean en `not_empty_`. Tiempo total dominado por la producción: $N \times 300 \text{ ms} = 9.0 \text{ s}$. Medido: 9.00 s.

La ratio observada ($\approx 2.81\times$) es consistente con la teórica ($3\times$). Confirma que el tiempo total lo fija el componente más lento, no el más rápido — exactamente el comportamiento que se espera de un *pipeline* limitado por su etapa más lenta.

El reparto **10 / 10 / 10** entre los tres consumidores en ambos casos es revelador y se discute en §4.2.

4. Preguntas de reflexión

4.1 Escalabilidad

¿Cómo cambia el rendimiento del sistema cuando aumentamos el número de consumidores? ¿Cómo afecta la proporción entre productores y consumidores al rendimiento general y a la probabilidad de bloqueo o inanición?

Con un único productor, el throughput agregado del sistema está acotado por $\min(\text{prod_rate}, n_{\text{consumers}} \times \text{cons_rate})$. Aumentar consumidores ayuda solo mientras el lado consumidor sea el cuello de botella. En cuanto se iguala o se supera al productor, añadir más consumidores no aporta nada: simplemente los hilos adicionales se bloquean en `not_empty_` con mayor frecuencia. Por ejemplo, en el caso 1 ya estamos justo en el punto de balance; un cuarto consumidor con `cons-sleep = 300 ms` no acortaría el tiempo total (el sistema pasaría a estar *producer-limited*).

El comportamiento contrasta con el del lector-escritor (no implementado, pero relevante a efectos comparativos): si los lectores fueran totalmente concurrentes y leyeran cada uno *toda* la secuencia escrita (modelo broadcast), añadir lectores no aceleraría el procesamiento de un escrito — solo aumentaría el trabajo paralelo en lectura. Si en cambio el lector-escritor compartiera la política clásica de Courtois (lectores concurrentes con exclusión frente al escritor), añadir lectores escalaría la lectura *mientras* no apareciera un escritor.

En cuanto a la **proporción**, dos extremos son interesantes:

- **Pocos productores, muchos consumidores** (mi caso, 1:3): la producción es el techo. El buffer tiende a vaciarse y la mayoría de consumidores espera la mayor parte del tiempo. No hay inanición porque cada item se asigna a un único consumidor y los consumidores van turnándose (ver §4.2).
- **Muchos productores, pocos consumidores**: el buffer tiende a saturarse y los productores se bloquean con frecuencia. Tampoco hay inanición estricta de productores en `notify_one` con un único consumidor, pero el throughput está acotado por el consumidor.

4.2 Equidad e inanición

¿Existen situaciones de inanición? ¿Qué mecanismo garantiza que todos los lectores lean los mismos datos?

En el productor-consumidor implementado, **cada ítem se entrega a exactamente un consumidor** (`notify_one` despierta a uno de los hilos en espera). No hay competición por el mismo dato y, en consecuencia, no hay inanición de consumidores en el sentido estricto: ninguno depende de un recurso que otro pueda monopolizar. El único hilo que puede quedarse esperando indefinidamente es uno bloqueado en `not_empty_` si la producción se detuviera sin invocar `mark_done()` — pero eso es un fallo de diseño, no un problema de la primitiva.

El `std::condition_variable::notify_one` **no garantiza fairness por contrato**: el estándar permite elegir cualquier hilo en espera. Aun así, los datos experimentales muestran un reparto de **10 / 10 / 10** entre los 3 consumidores en ambos casos. Esto se debe a que (a) los tres consumidores trabajan con el mismo `cons_sleep`, por lo que llegan a `remove()` aproximadamente desfasados un tercio del periodo; y (b) la implementación de `std::condition_variable` sobre futex del kernel Linux con NPTL tiende a despertar al hilo que lleva más tiempo bloqueado (FIFO en la práctica). Sin embargo, esta justicia es **emergente**, no contractual; un escenario con sleeps distintos por consumidor o cargas asimétricas podría sesgar el reparto.

La segunda parte de la pregunta — *qué mecanismo garantiza que todos los lectores reciban los mismos datos* — **no aplica al productor-consumidor**: el modelo es de consumo destructivo, cada elemento del buffer existe para ser extraído por un único hilo. En la adaptación broadcast del lector-escritor que plantea el enunciado, el mecanismo equivalente requeriría que cada slot del buffer mantuviera un *bitset* o contador de “lectores que ya han leído este slot”; el `front` solo avanzaría cuando los tres lectores hubiesen marcado el slot, y los lectores se sincronizarían con el escritor mediante una barrera (o una variable de condición sobre el contador de lecturas pendientes). Esa disciplina sí garantiza que los tres lectores procesen exactamente la misma secuencia, algo imposible con la API actual de `Buffer::remove()`.

4.3 Tiempo de espera

¿Qué impacto tienen los distintos tiempos de espera entre productor y consumidores en la eficiencia del sistema?

Los datos de §3 muestran el efecto cuantitativo:

- Cuando el productor es más rápido que los consumidores agregados, el sistema está **consumer-bound**: el buffer tiende a llenarse, el productor se bloquea en `not_full_` y el throughput lo fija el lado consumidor. La capacidad del buffer actúa de amortiguador: con `buffer = 5` el productor puede adelantarse hasta 5 ítems, pero no más.
- Cuando el productor es más lento, el sistema es **producer-bound**: el buffer está casi siempre vacío, los consumidores se bloquean en `not_empty_` y el throughput está acotado por la producción.

La eficiencia *teórica* del sistema (aprovechamiento del paralelismo) es máxima cuando ambos lados están balanceados — es justamente el caso 1 con la configuración 1 productor / 3 consumidores y razón de sleeps 100 : 300. En ese punto, ni el productor ni los consumidores se bloquean significativamente y el wall-clock se aproxima al ideal de un pipeline sin holguras.

Un detalle de implementación relevante: el cómputo de la operación (`restar_y_calcular_moda`, etc.) ocurre **fuera del mutex del buffer**. Cada consumidor extrae el valor con `remove()`, libera el lock y luego procesa. Eso significa que dos consumidores pueden computar simultáneamente (aunque sea sobre copias propias del vector) sin serializarse mutuamente. La única serialización entre consumidores es por la salida (`cout_mtx`) y por el acceso al monitor (`m_`), ambos de muy corta duración.

4.4 Rendimiento y sincronización de las operaciones

¿Cómo impactan las distintas operaciones (moda, desviación, sumatoria) en el tiempo de procesamiento y la sincronización del sistema? ¿Hay operaciones más propensas a causar retrasos?

Complejidad asintótica de cada operación (N = tamaño del vector = 100):

Consumidor	Operación	Complejidad	Muta el vector
C1	Resta + moda	$O(N \log N)$	Sí
C2	Promedio + desviación	$O(N)$	Sí
C3	Sumatoria · leído	$O(N)$	No

La moda domina por la ordenación del mapa de frecuencias (la suma y el conteo son lineales, pero el `std::sort` sobre los pares (valor, frecuencia) es $O(K \log K)$ con $K \leq N$). En el peor caso (todos los valores distintos), $K = N$ y el coste total es $O(N \log N)$.

Sin embargo, **para $N = 100$ la diferencia es irrelevante** frente a los sleeps de 100–300 ms: el cómputo de cada operación está en el orden de los microsegundos, mientras que el sleep está en milisegundos. Es decir, en este régimen ninguna operación causa retrasos perceptibles ni desplaza el cuello de botella. Si N creciese a varios miles de elementos, la moda empezaría a notarse y podría desincronizar al consumidor 1 respecto a los otros dos — acumularía menos items que C2 y C3 porque tardaría más en volver a llamar a `remove()`.

Hay un matiz de sincronización específico: las operaciones de C1 y C2 son **destructivas** (modifican el vector in-place). Por eso el enunciado pide explícitamente que cada consumidor mantenga su propia copia. Si compartiesen una sola copia del vector, harían falta un mutex adicional o políticas copy-on-write para evitar carreras. La decisión de copiar el vector inicial en el constructor de cada Consumer resuelve el problema de forma trivial al precio de $N \times n_consumers$ enteros de memoria ($3 \times 100 \times 4 \text{ B} \approx 1.2 \text{ KB}$, despreciable). Valgrind confirma la limpieza: 260 allocs / 260 frees, 0 bytes en uso a la salida.

5. Conclusiones

1. La implementación clásica del productor-consumidor con monitor (mutex + dos condition_variable con predicado) cumple el enunciado con código reducido, sin centinelas mágicos en el flujo de datos y con terminación limpia.
2. El tiempo total de la simulación está dictado por el componente más lento ($\max(1/\text{prod_rate}, N_{\text{cons}}/\text{cons_rate})$), no por el más rápido. El tamaño del buffer solo afecta a la varianza de la ocupación, no a la cota asintótica.
3. La equidad emergente entre los 3 consumidores (reparto 10/10/10) es un subproducto del scheduler de Linux y de la simetría de los sleeps, no una garantía de la primitiva `notify_one`.
4. Las tres operaciones son irrelevantes en coste para $N = 100$. La asimetría asintótica de la moda solo se manifestaría con vectores mucho más grandes.
5. Valgrind reporta 0 leaks, lo que confirma la correcta gestión de `std::thread` (todos los hilos se hacen `join`), de los locks (`std::lock_guard RAII`) y del optional de retorno.

Apéndice — reproducir los datos

```
cd productor-consumidor
make                               # compila prodcons
make run-caso1                     # caso 1 (consumer-bound)
```

```
make run-caso2          # caso 2 (producer-bound)
make valgrind           # genera docs/valgrind.log
```

Para medir wall-clock:

```
/usr/bin/time -f "wall=%es" ./prodcons --items 30 --buffer 5 \
  --prod-sleep 100 --cons-sleep 300 > /dev/null
```